

# Лабораторная работа №3 “Реализация формирования линейного кода в терминах набора инструкций абстрактной виртуальной машины”

## Цель работы

Реализовать формирование линейного кода в терминах набора инструкций абстрактной виртуальной машины для набора подпрограмм путем анализа графа потока управления.

## Описание реализации

### Структуры данных

Реализованы следующие структуры данных для представления информации об элементах образа программы:

1. **VMRegister** - перечисление типов регистров (R0-R7, IP, SP, BP, FLAGS)
2. **Operand** - структура операнда с поддержкой типов: регистр, немедленное значение, память, метка
3. **VMInstruction** - структура инструкции с массивом операндов
4. **VMDataItem** - структура элемента данных
5. **VMLabel** - структура метки с именем и адресом
6. **VMProgram** - структура образа программы (инструкции, данные, метки)

### Специализированные структуры для генерации

1. **RegisterAllocator** - отслеживает использование регистров R0-R7
2. **LocalVarMapping** - связывает имя локальной переменной с её офсетом и регистром
3. **ArgumentMapping** - связывает имя аргумента с его индексом и регистром
4. **FunctionContext** - контекст генерации кода для одной функции

### Алгоритм построения образа программы

Основной алгоритм реализован в функции `generate_function_code`:

1. **Генерация пролога функции:** PUSH BP, MOV BP, SP, SUB SP для локальных переменных
2. **Обход базовых блоков** в топологическом порядке:
  - Для каждого блока добавляется метка вида `func_block_label`
  - Если блок содержит операции типа “block”, генерируются все инструкции блока

- Проверяется, содержит ли блок управляющие конструкции (if/while)

### 3. Генерация successors:

- Для обычных и пустых блоков генерируются JMP к successor-ам
- Для блоков с if/while successors НЕ генерируются (они сами управляют переходами)

### 4. Топологическая сортировка блоков обеспечивает правильный порядок генерации

## Особенности реализации управления потоком

**Генерация successors** реализована с умным определением типов блоков:

```
static bool is_control_flow_statement(const char *type) {
    return (strcmp(type, "ifStmt") == 0 ||
            strcmp(type, "if") == 0 ||
            strcmp(type, "whileStmt") == 0 ||
            strcmp(type, "while") == 0);
}
```

- Обычные блоки (присваивания, varDecl)  $\square$  генерируют JMP к successors
- Блоки с if/while  $\square$  НЕ генерируют successors (управление внутри generate\_statement)
- Пустые блоки  $\square$  генерируют JMP к successors для связывания блоков

**Генерация условий** для операторов сравнения:

Для выражений вида  $a \text{ op } b$  генерируется оптимальный код без двойного CMP:

| Оператор | Логика прыжка                     | Инструкции       |
|----------|-----------------------------------|------------------|
| $a > b$  | Прыгать в else если $a \square b$ | JL else, JE else |
| $a < b$  | Прыгать в else если $a \square b$ | JG else, JE else |
| $a == b$ | Прыгать в else если $a \neq b$    | JNE else         |

## Результаты тестирования

### Пример 1: Рекурсивное вычисление чисел Фибоначчи

**Исходный код:**

```
// Вычисление числа Фибоначчи
// fib(1) = 1, fib(2) = 1, fib(n) = fib(n-2) + fib(n-1)
int fib(int n) {
    int r;
```

```

    if (n < 2) {
        r = 1;
    } else {
        r = fib(n - 2);
        r = r + fib(n - 1);
    }
    return r;
}

int main() {
    int result;
    result = fib(10);
}

```

### Сгенерированный ассемблерный код:

```

fib:
    PUSH BP                ; Сохраняем базовый указатель
    MOV BP, SP             ; Устанавливаем новый BP
    SUB SP, 8              ; Выделяем место под локальную переменную r
fib_entry_0:
    LOAD R0, [BP+8]        ; Загружаем аргумент n
    MOV R1, 2              ; Константа 2
    CMP R0, R1             ; Сравнение n < 2
    JL fib_then_2          ; Переход в if, если n < 2
    JMP fib_else_4         ; Иначе переход в else
    JMP fib_then_2
fib_then_2:
    MOV R1, 1              ; r = 1
    STORE [BP-4], R1
    JMP fib_merge_3
fib_merge_3:
    JMP fib_exit_1
fib_else_4:
    LOAD R1, [BP+8]        ; Рекурсивный вызов fib(n-2)
    MOV R2, 2
    SUB R1, R2
    PUSH R1
    CALL fib
    ADD SP, 4
    STORE [BP-4], R0       ; r = fib(n-2)
    LOAD R1, [BP+8]        ; Рекурсивный вызов fib(n-1)

```

```

MOV R2, 1
SUB R1, R2
PUSH R1
CALL fib
ADD SP, 4
LOAD R1, [BP-4]
ADD R1, R0          ; r = r + fib(n-1)
STORE [BP-4], R1
JMP fib_merge_3
fib_exit_1:
LOAD R0, [BP-4]    ; Загрузка возвращаемого значения
MOV SP, BP
POP BP
RET

```

**Анализ генерации:** - Правильно генерируется пролог функции с выделением памяти под локальные переменные - Условие  $n < 2$  корректно транслируется в последовательность CMP, JL, JMP - Рекурсивные вызовы fib(n-2) и fib(n-1) проходят через правильную последовательность LOAD, SUB, PUSH, CALL, ADD SP - Результат сложения аккумулируется в локальной переменной r - Эпилог функции корректно загружает возвращаемое значение и восстанавливает стек

---

## Пример 2: Калькулятор с множеством условий в цикле

### Исходный код:

```

void writeByte(byte b);
byte readByte();

void main() {
    int acc = 0;
    byte op;
    byte digit;
    int operand;
    byte zero = "0";

    while (1) {
        writeByte(12 + zero);
        writeByte(acc + zero);
        op = readByte();
    }
}

```

```

digit = readByte();

operand = digit - zero;

if (op == "+") {
    acc = acc + operand;
}
if (op == "-") {
    acc = acc - operand;
}
if (op == "*") {
    acc = acc * operand;
}
if (op == "/") {
    acc = acc / operand;
}
if (op == zero) {
    break;
}
}
}

```

#### **Фрагмент сгенерированного кода (часть условий вычислений):**

```

main_while_body_3:
    ; acc = acc + operand (при op == '+')
    LOAD R3, [BP-4]      ; Загрузка acc
    LOAD R4, [BP-16]     ; Загрузка operand
    ADD R3, R4           ; Сложение
    STORE [BP-4], R3     ; Сохранение результата в acc
    JMP main_merge_6

main_merge_6:
    LOAD R2, [BP-8]      ; Загрузка op для проверки '-'
    MOV R5, 45           ; ASCII код '-'
    CMP R2, R5           ; Сравнение op == '-'
    JE main_then_7
    JMP main_merge_8

main_then_7:
    LOAD R3, [BP-4]      ; Загрузка acc
    LOAD R4, [BP-16]     ; Загрузка operand

```

```

SUB R3, R4          ; Вычитание
STORE [BP-4], R3    ; Сохранение результата
JMP main_merge_8

```

main\_merge\_8:

```

LOAD R2, [BP-8]     ; Загрузка op для проверки '*'
MOV R5, 42           ; ASCII код '*'
CMP R2, R5           ; Сравнение op == '*'
JE main_then_9
JMP main_merge_10

```

main\_then\_9:

```

LOAD R3, [BP-4]     ; Загрузка acc
LOAD R4, [BP-16]    ; Загрузка operand
MUL R3, R4           ; Умножение
STORE [BP-4], R3     ; Сохранение результата

```

**Анализ генерации:** - Правильная обработка символьных литералов (например, “0” преобразуется в ASCII код 48) - Последовательные операторы if корректно транслируются в цепочку проверок CMP, JE, JMP - Каждый оператор (+, -, \*, /) генерируется с соответствующей VM инструкцией - Цикл while(1) правильно реализуется с условием на истинность (JNE на метку цикла) - Арифметические выражения с несколькими операндами корректно вычисляются через временное значение в регистрах

---

### Пример 3: Вложенные условия внутри цикла

**Исходный код:**

```

int compute(int x) {
    int i;
    int result;
    i = 0;
    result = 0;

    while (i < x) {
        if (i > 5) {
            result = result + i;
        } else {
            if (i < 2) {
                result = result + 1;
            }
        }
        i++;
    }
}

```

```

        } else {
            result = result + 2;
        }
    }
    i = i + 1;
}
return result;
}

```

```

int main() {
    int a;
    int b;
    a = 10;
    b = compute(a);

    if (b > 20) {
        final = b - 10;
    } else {
        final = b + 5;
    }
}

```

### **Сгенерированный ассемблерный код (ключевые фрагменты):**

```

compute_while_body_3:
    LOAD R0, [BP-4]      ; if (i > 5)
    MOV R2, 5
    CMP R0, R2
    JG compute_then_5
    JMP compute_else_7
    JMP compute_then_5

compute_then_5:
    LOAD R2, [BP-8]      ; result = result + i
    LOAD R0, [BP-4]
    ADD R2, R0
    STORE [BP-8], R2
    JMP compute_merge_6

compute_else_7:
    LOAD R0, [BP-4]      ; if (i < 2)
    MOV R3, 2

```

```

CMP R0, R3
JL compute_then_8
JMP compute_else_10
JMP compute_then_8

```

```

compute_then_8:
    LOAD R2, [BP-8]      ; result = result + 1
    MOV R3, 1
    ADD R2, R3
    STORE [BP-8], R2
    JMP compute_merge_9

```

```

compute_else_10:
    LOAD R2, [BP-8]      ; result = result + 2
    MOV R3, 2
    ADD R2, R3
    STORE [BP-8], R2
    JMP compute_merge_9

```

```

compute_merge_9:
    JMP compute_merge_6

```

```

compute_merge_6:
    LOAD R0, [BP-4]      ; i = i + 1
    MOV R3, 1
    ADD R0, R3
    STORE [BP-4], R0
    JMP compute_while_cond_2

```

**Анализ генерации:** - Вложенные условия корректно транслируются с использованием нескольких меток (then\_X, else\_X, merge\_X) - Трёхуровневая вложенность (внешний if [?] внешний else [?] внутренний if) правильно обрабатывается - Общая точка слияния compute\_merge\_6 используется после завершения внутренней логики - Оптимизация условий для операторов сравнения: JG для ">" и JL для "<", без лишних сравнений - Цикл правильно реализуется с возвратом к метке условия compute\_while\_cond\_2 - Все пути в графе потока управления корректно связываются инструкциями JMP

## Выводы

Разработанный модуль успешно генерирует линейный код из графов потока управления. Ключевые достижения:

1. **Правильная генерация successors:** обычные блоки связываются JMP, управляющие



конструкции не создают лишних переходов

2. **Оптимизация условий:** генерация прямых переходов без двойного сравнения для операторов сравнения
3. **Поддержка вложенных конструкций:** корректная работа со сложными случаями (циклы с условиями внутри)
4. **Работа с CFG:** поддерживаются как типы из AST (ifStmt, whileStmt), так и из CFG (if, while)

Модуль готов к использованию и может быть расширен дополнительными инструкциями ВМ и оптимизациями.